

实验五：用户态虚拟内存管理器 参考手册

1. 虚拟内存与按需分配

进程看到的是虚拟地址空间。虚拟地址空间可以很大，但并不意味着对应的物理内存已经全部分配。

Linux 常见行为：

- `mmap` 可以先建立一段虚拟内存区域；
- 页面在真正被访问时才触发缺页；
- 内核在缺页路径中分配物理页、更新页表；
- 进程的 `VRT` 代表虚拟地址空间规模；
- 进程的 `RES/RSS` 更接近实际驻留在物理内存中的页。

本实验的前置观察任务会同时做两件事：一方面通过“先申请、后逐页写入”观察 `VRT` 与 `RES/RSS` 的差异；另一方面通过 `pagemap` 观察某个虚拟页在访问前后的页表项状态变化。

2. mmap 与 munmap

`mmap` 可以向内核申请一段虚拟地址空间：

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

2.1 参数含义

`mmap` 的 6 个参数都必须填写，不能省略。但不同使用场景下，有些参数可以使用固定值。

参数	是否必填	含义	本实验中的常用取值
<code>addr</code>	必填	希望映射到哪个虚拟地址。如果填 <code>NULL</code> ，表示不指定地址，让内核自动选择。	<code>NULL</code>
<code>length</code>	必填	映射区域的字节数。通常应按页大小的整数倍理解，内核会按页管理。	任务需要的字节数
<code>prot</code>	必填	映射区域的访问权限。可以控制是否允许读、写、执行。	<code>PROT_READ</code> <code>PROT_WRITE</code> 或 <code>PROT_NONE</code>
<code>flags</code>	必填	映射类型和共享方式。决定这段映射是否关联文件、是否与其他进程共享修改等。	<code>MAP_PRIVATE</code> <code>MAP_ANONYMOUS</code>
<code>fd</code>	必填	文件映射使用的文件描述符。匿名映射不依赖文件。	匿名映射时填 <code>-1</code>
<code>offset</code>	必填	文件映射从文件的哪个偏移开始。匿名映射不使用该值。	匿名映射时填 <code>0</code>

也就是说，在 C 语言函数调用层面，这 6 个参数都必须写出来；只是匿名映射场景下，`fd` 和 `offset` 没有实际文件含义，通常固定写成 `-1` 和 `0`。

2.2 prot：页面权限

`prot` 决定映射出来的页面允许哪些访问。常见取值包括：

- `PROT_NONE`：不可读、不可写、不可执行。访问这类页面通常会触发 `SIGSEGV`。
- `PROT_READ`：允许读取。
- `PROT_WRITE`：允许写入。
- `PROT_EXEC`：允许执行机器指令。

多个权限可以用按位或 `|` 组合。例如：

```
PROT_READ | PROT_WRITE
```

表示该页面既允许读，也允许写。本实验的前置观察任务需要可读写页面；`mini_vmem` 任务中，程序初始会用 `PROT_NONE` 制造缺页，再在缺页处理时改成可读写。

2.3 flags：映射类型

`flags` 决定映射的类型。常见标志包括：

- `MAP_PRIVATE`：私有映射。对映射区域的修改不会写回原文件，也不会被其他进程共享看到。匿名内存通常使用它。
- `MAP_SHARED`：共享映射。多个进程映射同一文件时，修改可以对其他映射者可见，也可能写回文件。
- `MAP_ANONYMOUS`：匿名映射。映射不来自任何文件，通常用于申请一段由内核提供的零初始化内存。

多个 `flags` 同样用 `|` 组合。例如：

```
MAP_PRIVATE | MAP_ANONYMOUS
```

表示申请一段私有匿名内存。因为它不关联文件，所以 `fd` 通常填 `-1`，`offset` 填 `0`。

2.4 返回值与错误处理

`mmap` 成功时返回映射区域的起始地址；失败时返回 `MAP_FAILED`，不是返回 `NULL`。因此应使用：

```
if (ptr == MAP_FAILED) {
    perror("mmap failed");
}
```

常见错误包括权限组合不合法、文件描述符无效、长度为 0、系统资源不足等。

2.5 munmap：释放映射区域

`mmap` 成功后，内核会把一段虚拟地址范围加入当前进程的地址空间。程序不再需要这段区域时，应使用 `munmap` 解除映射：

```
int munmap(void *addr, size_t length);
```

这里两个参数应和前面成功的 `mmap` 对应：

- `addr`：要释放的映射起始地址，也就是 `mmap` 成功返回的指针；
- `length`：要释放的长度，通常就是当初传给 `mmap` 的长度。

在前置观察任务中，`mmap` 返回值保存在 `ptr`，申请长度保存在 `size`，因此释放时应围绕这两个变量思考。释放成功后，这段虚拟地址范围不再属于当前进程；继续读写这段地址通常会触发段错误。`munmap` 成功返回 `0`，失败返回 `-1`，实际工程代码中也应检查返回值。

本实验会在释放后再次暂停，让你继续观察同一个进程。如果 `munmap` 写对，`VIRT` 通常会减少接近前面申请的大小，`RES/RSS` 也会释放已经驻留的页面；如果没有真正调用 `munmap`，这些数值不会出现对应下降。

页大小可以通过：

```
long page_size = sysconf(_SC_PAGESIZE);
```

常见 Linux 系统页大小是 4096 字节，但代码中不要写死，应通过 `sysconf` 获取。

3. pagemap 的读取方式

Linux 通过 `/proc/[pid]/pagemap` 暴露进程虚拟页的映射信息。可以把这个文件理解成一个“页表项数组”：每个虚拟页对应一个 64 位条目，也就是 8 字节。

3.1 什么是虚拟页号 VPN

虚拟地址不是直接拿来索引 `pagemap` 的。操作系统按“页”管理内存，所以需要先判断某个虚拟地址属于第几页。这个“第几页”就是 **VPN (Virtual Page Number, 虚拟页号)**。

可以先用一个小例子理解。假设页大小是 4096 字节，那么虚拟地址会被分成一页一页：

```
第 0 页：地址 0 到 4095
第 1 页：地址 4096 到 8191
第 2 页：地址 8192 到 12287
第 3 页：地址 12288 到 16383
```

所以：

地址 0	属于第 0 页, 页内位置是 0
地址 100	属于第 0 页, 页内位置是 100
地址 4095	属于第 0 页, 页内位置是 4095
地址 4096	属于第 1 页, 页内位置是 0
地址 5000	属于第 1 页, 页内位置是 904
地址 8192	属于第 2 页, 页内位置是 0

换成十六进制也一样。4096 字节等于 `0x1000`, 因此:

地址 <code>0x0000</code> 到 <code>0x0fff</code>	属于第 0 页
地址 <code>0x1000</code> 到 <code>0x1fff</code>	属于第 1 页
地址 <code>0x2000</code> 到 <code>0x2fff</code>	属于第 2 页
地址 <code>0x3000</code> 到 <code>0x3fff</code>	属于第 3 页

例如地址 `0x12345` 落在 `0x12000` 到 `0x12fff` 这一页中。它前面已经跨过了多少个完整的 4096 字节页面, 这个数量就是 VPN; 它距离该页起始地址 `0x12000` 有多远, 就是页内偏移。

在 C 代码中, 虚拟地址是一个指针。要做页号计算时, 通常需要先把指针转换成能参与整数运算的类型, 例如 `uintptr_t`。至于如何从地址得到“跨过了多少个完整页面”和“在页内走了多远”, 请根据上面的页范围例子自行推导。

常用字段:

- bit 63: Present, 页面当前是否驻留在物理内存中;
- bit 62: Swapped, 页面是否被换出;
- bit 0-54: PFN, 物理页帧号。

注意: 这里的 bit 编号从最低位开始。bit 0 是最右边的最低位, bit 63 是 64 位整数中最左边的最高位。例如 `0x8000000000000000` 只有最高位为 1, 因此它的 bit 63 为 1。

PFN (Physical Frame Number) 可以理解为“物理页帧编号”。操作系统不仅把虚拟内存按页切分, 也会把物理内存按同样大小切成一块一块的页帧。如果页大小是 4096 字节, 那么物理内存也可以想象成:

物理页帧 0:	物理地址 0 到 4095
物理页帧 1:	物理地址 4096 到 8191
物理页帧 2:	物理地址 8192 到 12287
...	

页表项里的 PFN 说明: 这个虚拟页当前映射到了哪一个物理页帧。它不是完整物理地址, 只是物理页帧的编号。真正访问某个字节时, 还需要加上该虚拟地址在页内的位置。

例如某个虚拟地址位于自己的虚拟页内偏移 `0x123`, `pagemap` 显示它映射到物理页帧 `0x50`, 那么可以理解为: 这个虚拟页的内容放在物理内存第 `0x50` 个页帧中, 而具体字节在该页帧内部偏移 `0x123` 的位置。

3.2 条目和文件偏移是什么

这里的“条目”指 `pagemap` 文件中保存的一个 64 位记录。它不是一个 C 结构体名, 而是 Linux 约定的二进制记录格式。

可以这样理解 `pagemap` 文件:

```
文件字节 0 到 7 : 第 0 个虚拟页的 pagemap 条目
文件字节 8 到 15 : 第 1 个虚拟页的 pagemap 条目
文件字节 16 到 23 : 第 2 个虚拟页的 pagemap 条目
文件字节 24 到 31 : 第 3 个虚拟页的 pagemap 条目
...
```

所以，当你已经知道某个地址属于第几个虚拟页后，就可以去 pagemap 文件中找到“第几个条目”。`lseek` 需要的 `offset` 是这个条目在 pagemap 文件中的字节位置。

注意不要混淆两个不同的偏移：

- **页内偏移**：某个虚拟地址在它所属页面内部的位置，例如地址 5000 在第 1 页内的位置是 904。
- **pagemap 文件偏移**：为了读取某个虚拟页的 8 字节条目，需要在 `/proc/self/pagemap` 文件中跳到的位置。

给定虚拟地址 `addr`，读取对应 pagemap 条目的步骤：

1. 获取页大小 `page_size`；
2. 将地址转换为整数类型，如 `uintptr_t`；
3. 计算虚拟页号 `vpn`；
4. 根据 `vpn` 找到它在 pagemap 文件中对应的 8 字节条目位置；
5. 使用 `lseek` 把文件当前位置移动到该条目开头；
6. 使用 `read` 从当前位置读取 8 字节，得到该虚拟页的原始 pagemap 条目；
7. 用位运算解析 Present 位和 PFN。

`Present` 是 pagemap 条目中的一个状态位，表示这个虚拟页当前是否已经有有效映射。它只能说明“页表项当前是否 present”，不能单独解释为什么 present 或 not present；原因还需要结合页面是否访问过、权限、内核安全限制等现象一起分析。

由于真实 PFN 会暴露物理内存布局，现代 Linux 通常要求 root 权限才能看到有效 PFN，因此运行 pagemap 程序时一般需要 `sudo`。

3.3 htop 中的 VIRT 与 RES

运行 `vm_observe` 时，程序会先打印自己的 PID，并在调用 `mmap` 前暂停一次。观察 `htop` 时应使用这个 PID 启动，例如程序打印 `vm_observe pid = 13995`，就新开终端运行：

```
sudo htop -p 13995
```

这样 `htop` 只显示目标进程。`htop` 里看到的 `VIRT` 和 `RES` 不是只统计本实验里这一次 `mmap` 出来的区域，而是整个进程当前的内存统计。

- `VIRT`：进程当前拥有的全部虚拟地址空间，包括程序代码段、动态库、堆、栈、`mmap` 区域等。
- `RES`：当前实际驻留在物理内存中的页面总量，也包括程序代码、动态库、栈、已经访问过的数据页等。

因此，程序刚启动时，即使还没有执行本实验中的大块 `mmap`，`VIRT` 和 `RES` 也都会有值。

- `VIRT` 已经包含程序本身、动态链接器、C 标准库、堆、栈等映射到进程地址空间的区域。
- `RES` 已经包含当前真正驻留在物理内存里的页面，例如正在执行的代码页、部分动态库页面和栈页面。

所以阶段 0 看到的不是“空进程”，而是一个已经完成基本加载、正在运行并等待输入的普通 Linux 进程。它还没有申请那段 512MB 实验内存，但它已经拥有自己的地址空间，也已经有一部分页面驻留在物理内存中。

调用 `mmap` 申请 512MB 后，`VIRT` 通常会明显增加，因为进程获得了一段新的虚拟地址范围。但如果此时还没有访问这段内存，`RES` 通常不会同步增加 512MB。原因是匿名映射采用按需分配：`mmap` 先建立虚拟地址范围，真正的物理页通常要等到读写该地址时才分配。

写入首个页面时，理论上内核会为此虚拟页准备实际物理页面，因此 `RES` 可能增加一个页左右。常见页大小是 4096 字节，也就是 4KB。这个变化相对于整个进程已有的 `RES` 很小，在 `htop` 中可能看起来“没有增加”，常见原因包括：

- `htop` 的显示单位或取整方式让 4KB 变化不明显；
- `htop` 刷新间隔较长，观察时刻没有正好捕捉到变化；
- 进程其他页面的驻留状态也可能同时有小幅波动，掩盖单页变化；
- 有些终端宽度或列设置会让数值显示得更粗略。

因此，阶段 2 写入首个页面后，重点不是要求 `RES` 肉眼可见地大幅增加，而是理解：只写一个页最多只带来很小的驻留内存变化。等到阶段 3 按页逐步写完整段 512MB 内存时，`RES` 才会随着大量页面被实际访问而明显增长。

4. `mprotect` 与页面权限

`mprotect` 用来修改一段已有虚拟内存区域的访问权限：

```
int mprotect(void *addr, size_t len, int prot);
```

常见权限：

- `PROT_NONE`：不可读、不可写、不可执行；
- `PROT_READ`：可读；
- `PROT_WRITE`：可写；
- `PROT_READ | PROT_WRITE`：可读写。

操作系统按“页”管理权限，因此传给 `mprotect` 的起始地址需要按页边界对齐。若某个故障地址落在页面中间，通常需要先计算它所在页面的起始地址，再修改整页权限。

地址对齐常用思路：

```
uintptr_t page_base = address & ~(page_size - 1);
```

这个写法依赖页大小是 2 的幂。Linux 常见页大小满足这一条件。

5. `SIGSEGV` 与 `sigaction`

当程序访问无权限页面，例如访问 `PROT_NONE` 页面时，CPU 会触发异常，Linux 会向进程发送 `SIGSEGV`。

使用 `sigaction` 注册处理函数：

```
struct sigaction sa;
memset(&sa, 0, sizeof(sa));
sa.sa_flags = SA_SIGINFO;
sa.sa_sigaction = handler;
sigemptyset(&sa.sa_mask);
sigaction(SIGSEGV, &sa, NULL);
```

这几行的作用可以拆开理解：

- `struct sigaction sa`：准备一个结构体，用来描述某个信号应该怎样处理。
- `memset(&sa, 0, sizeof(sa))`：先清零，避免结构体中未初始化的字段影响行为。
- `sa.sa_flags = SA_SIGINFO`：选择三参数形式的处理函数，让 handler 能拿到 `siginfo_t *info`。
- `sa.sa_sigaction = handler`：指定真正要调用的处理函数。
- `sigemptyset(&sa.sa_mask)`：handler 运行期间不额外屏蔽其他信号。
- `sigaction(SIGSEGV, &sa, NULL)`：把这套规则安装到 `SIGSEGV` 上。

安装完成后，本进程再次因为访问 `PROT_NONE` 页面而收到 `SIGSEGV` 时，Linux 会调用你注册的 handler。对于 `mini_vmem` 来说，这一步相当于把“访问不可访问页面后直接崩溃”的默认行为，改成“进入用户态缺页处理函数”。

带 `SA_SIGINFO` 的 handler 形式：

```
void handler(int sig, siginfo_t *info, void *context);
```

其中：

```
info->si_addr
```

表示导致异常的虚拟地址。

如果 handler 修复了页面权限并返回，处理器会回到异常发生的位置继续执行。对于缺页类异常，这通常表现为导致缺页的内存访问指令被再次尝试执行。

6. 用户态页表设计

`mini_vmem` 用一个数组模拟页表。每个页表项记录一个虚拟页当前处于什么状态。

推荐状态：

```
typedef enum {
    PAGE_EMPTY = 0,
    PAGE_RESIDENT,
    PAGE_SWAPPED
} page_state_t;
```

状态含义：

- `PAGE_EMPTY`：页面从未装入；
- `PAGE_RESIDENT`：页面当前驻留在内存中；

- `PAGE_SWAPPED`: 页面内容已保存在 swap 文件中, 当前不在内存中。

推荐页表项:

```
typedef struct {
    page_state_t state;
    int dirty;
    unsigned long load_time;
} page_entry_t;
```

- `dirty` 表示页面内容是否需要写回;
- `load_time` 可用于 FIFO 页面置换。

7. 地址到虚拟页号

`mini_vmem` 会用 `mmap` 申请一段连续虚拟内存, 记为 `vm_base`。

若管理区共有 `NUM_VPAGES` 页, 每页大小为 `page_size`, 则合法地址范围是:

```
[vm_base, vm_base + NUM_VPAGES * page_size)
```

地址转页号的基本过程:

1. 将 `vm_base` 和目标地址转换为 `uintptr_t`;
2. 判断目标地址是否在合法范围内;
3. 计算偏移量 `target - start`;
4. 用偏移量除以 `page_size` 得到虚拟页号。

页起始地址可以由页号反推:

```
page_start = vm_base + page_no * page_size
```

这里的页号和 `pagemap` 中的全局 VPN 有一点区别:

- `pagemap` 读取时, 需要判断某个地址在整个进程虚拟地址空间中属于第几页;
- `mini_vmem` 中, 需要判断某个地址在 `vm_base` 开始的这段实验区域中属于第几页。

举例: 假设 `vm_base = 0x10000000`, `page_size = 0x1000`, 故障地址 `addr = 0x10003123`。

从 `vm_base` 开始数:

```
第 0 页: 0x10000000 到 0x10000fff
第 1 页: 0x10001000 到 0x10001fff
第 2 页: 0x10002000 到 0x10002fff
第 3 页: 0x10003000 到 0x10003fff
```

地址 `0x10003123` 落在第 3 页范围内。它所在页的起始地址是 `0x10003000`, 它在这一页内部的位置是 `0x123`。实现 `page_of_addr()` 时, 本质上就是让程序自动完成这类判断。

8. FIFO 页面置换

当驻留页数量达到上限，而程序又访问一个不在内存中的页面时，需要选择一个 victim 页面淘汰。

FIFO 策略的思想是：

- 页面装入内存时记录装入时间；
- 每次淘汰当前驻留页面中装入时间最早的页面；
- 页面被再次访问不改变它的 FIFO 装入时间。

实现时要注意只在 `PAGE_RESIDENT` 页面中选择 victim，不能选择还没装入或已经换出的页面。

9. swap 文件设计

当物理内存不够用时，操作系统不能简单丢掉某个仍然有用的页面内容。常见做法是先把暂时不用的页面内容写到磁盘上的一块区域中，等以后再次访问这个页面时，再从磁盘读回内存。这块用于临时保存被换出页面内容的磁盘空间，通常称为 **swap 区** 或 **交换区**。

可以把 swap 区理解成“内存页面的临时仓库”：

- 页面在内存中时，可以直接被 CPU 读写；
- 页面被淘汰时，如果内容还需要保留，就把整页内容写入 swap 区，这个过程常叫 **swap-out**；
- 页面以后再次被访问时，再把它从 swap 区读回内存，这个过程常叫 **swap-in**。

真实 Linux 系统中的 swap 区可能是专门的磁盘分区，也可能是系统配置的 swap 文件。本实验不直接操作系统真实 swap，而是使用一个普通文件模拟交换区，例如 `mini_vmem.swap`。这样做的好处是逻辑清楚、不会影响系统配置，也方便你通过代码看到“页面写出”和“页面读回”的过程。

初始化时创建并截断文件：

```
int fd = open("mini_vmem.swap", O_CREAT | O_RDWR | O_TRUNC, 0600);
ftruncate(fd, page_size * NUM_VPAGES);
```

这两行分别做不同的事：

- `open` 负责创建或打开文件。`O_CREAT` 表示文件不存在时创建，`O_RDWR` 表示后续既要写页面也要读页面，`O_TRUNC` 表示每次运行都清空旧内容，`0600` 表示只有当前用户可读写。
- `ftruncate` 负责把文件大小调整为 `page_size * NUM_VPAGES`。可以把这个文件想成一排固定大小的格子，每个格子大小都是 `page_size`。`NUM_VPAGES` 个虚拟页，就需要 `NUM_VPAGES` 个格子。`ftruncate` 只是把文件扩展到足够大，划出这些格子；它不代表页面内容已经被写进去了。

一种简单设计是给每个虚拟页分配固定槽位：

```
第 0 页 -> 文件偏移 0
第 1 页 -> 文件偏移 page_size
第 2 页 -> 文件偏移 2 * page_size
...
第 page_no 页 -> 文件偏移 page_no * page_size
```

例如页大小为 4096 字节时，第 0 页保存在文件 [0, 4096) 这段范围，第 1 页保存在 [4096, 8192)，第 2 页保存在 [8192, 12288)。这样换出或换入某一页时，程序只要根据页号计算文件偏移，就能找到这页对应的位置。

写出页面时：

1. `lseek` 到对应偏移；
2. 写入整页内容。

读回页面时：

1. `lseek` 到同一偏移；
2. 读回整页内容。

`lseek` 用来移动文件的当前读写位置。它的常用形式是：

```
off_t lseek(int fd, off_t offset, int whence);
```

在本实验中，`fd` 是 swap 文件描述符，`offset` 是当前虚拟页在 swap 文件中的起始字节位置，`whence` 使用 `SEEK_SET`，表示从文件开头开始计算偏移。

```
if (lseek(fd, offset, SEEK_SET) == (off_t)-1) {  
    perror("lseek");  
    exit(1);  
}
```

完成 `lseek` 后，下一次 `read` 或 `write` 就会从这个文件位置开始。也就是说，`lseek` 本身不读写数据，它只负责把文件位置移动到正确的 swap 槽位。

换出页面时，基本顺序是：

1. 根据被淘汰页号计算它在 swap 文件中的偏移；
2. 使用 `lseek` 移动到该偏移；
3. 使用 `write` 类函数写出完整一页。

换入页面时，基本顺序是：

1. 根据故障页号计算它在 swap 文件中的偏移；
2. 使用 `lseek` 移动到该偏移；
3. 使用 `read` 类函数读回完整一页。

如果省略 `lseek`，`read` 或 `write` 会从文件当前所在位置继续进行。这个位置可能是上一次读写留下的，不一定是当前页面对应的槽位，因此会破坏“虚拟页号固定对应 swap 槽位”的设计。

实际系统的 swap 管理会复杂得多，需要维护空闲槽位、回收策略和 I/O 调度。本实验采用固定槽位是为了把重点放在缺页处理和页面置换控制流上。

初始化最后还要调用 `install_handler()` 注册 `SIGSEGV` 处理函数。因为虚拟内存区一开始被设置为 `PROT_NONE`，第一次访问任意页面都会触发 `SIGSEGV`。注册 handler 后，这个信号会进入 `segv_handler()`，由实验代码完成装页、换入或淘汰；如果没有注册 handler，程序第一次访问 `PROT_NONE` 页面时就会直接崩溃。

装入目标页时，需要先把该页权限恢复为 `PROT_READ | PROT_WRITE`。因为页面当前是 `PROT_NONE`，如果不先调用 `mprotect`，后面的清零或读入操作本身也会再次触发 `SIGSEGV`。

权限恢复后，根据页表状态决定页面内容来源：

- 如果目标页是 `PAGE_EMPTY`，说明它从未装入过，也没有保存在 swap 文件中的历史内容。此时可以使用 `memset` 将整页清零，模拟第一次分配到一页干净内存。
- 如果目标页是 `PAGE_SWAPPED`，说明它之前被淘汰过，有效内容保存在 swap 文件的固定槽位中。此时不能清零，而应从 `page_no * page_size` 对应的文件偏移读回整页内容。

`memset` 的常用形式是：

```
void *memset(void *s, int c, size_t n);
```

其中 `s` 是要写入的内存起始地址，`c` 是要填充的字节值，`n` 是要填充的字节数。清零一整页时，应让起始地址指向页起始位置，长度覆盖完整一页。

10. mini_vmem 的整体控制流

```
访问 vm_base 管理区域中的某个地址
|
v
页面当前为 PROT_NONE，触发 SIGSEGV
|
v
handler 从 info->si_addr 得到故障地址
|
v
计算虚拟页号
|
v
如果驻留页已满，选择 victim 并换出
|
v
如果目标页是 PAGE_EMPTY，清零装入
如果目标页是 PAGE_SWAPPED，从 swap 文件读回
|
v
修改目标页权限为可读写，更新页表
|
v
handler 返回，原访问继续执行
```

调试时建议在每次缺页、装入、淘汰时打印页号和 swap 偏移，这样可以确认状态变化是否符合预期。

11. Dirty Tracking

页面是否 dirty，表示它的内容相对后备存储是否被修改过。

本实验基础版本可以采用保守策略：页面装入后，只要允许写入，就把它视为 dirty。这样可能多写 swap，但不会丢失数据。

更精确的做法需要区分读访问和写访问。一种用户态模拟方式是：

1. 页面装入时只设置 `PROT_READ`;
2. 第一次写入触发权限异常;
3. handler 将页面改为 `PROT_READ | PROT_WRITE`;
4. 同时设置 dirty 位。

这种做法需要 handler 能区分“页面不在内存中”与“页面在内存中但写权限不足”两类缺页。

12. 进阶挑战提示

12.1 Second Chance 页面置换

Second Chance 可以看成 FIFO 的改进版。FIFO 只关心页面进入内存的先后顺序；Second Chance 还会给最近被访问过的页面一次“暂不淘汰”的机会。

真实硬件通常会在页表项中维护访问位。本实验运行在用户态，不能直接修改真实页表中的访问位。因此，如果程序继续直接用 `vm_base[index]` 读写页面，已经驻留的页面再次被访问时不会触发 handler，实验代码也就无法知道这个页面被访问过。

一种适合本实验的做法是把测试代码中的直接指针访问封装成访问接口，例如按页号或按字节偏移读写。访问接口负责两件事：

1. 完成实际读写；
2. 更新该页在用户态页表中的 `referenced` 信息。

这样，页面装入、页面读写和置换算法都通过同一份用户态页表协作，Second Chance 才能观察到“最近访问过”这个信息。

实现置换时，可以维护一个扫描位置，表示下一次从哪里开始寻找 victim。扫描到驻留页时，根据它的 `referenced` 信息决定是给它一次机会，还是选择它作为 victim。测试时应设计能体现差异的访问序列，例如让某些早进入内存的页面在淘汰前被再次访问，观察它们是否能避开一次 FIFO 淘汰。

12.2 写保护 Dirty Tracking

基础版把装入后可写的页面都保守视为 dirty，这样不会丢数据，但可能把只读过的页面也写回 swap。写保护 dirty tracking 的目标是区分“页面已经在内存中”和“页面确实被写过”。

关键思路是利用 `mprotect` 制造一次额外的权限异常：

1. 页面内容准备好后，先让该页保持只读；
2. 读访问可以正常完成，不需要设置 dirty；
3. 第一次写访问会因为权限不足触发 `SIGSEGV`；
4. handler 根据页表状态判断这不是普通缺页，而是写保护缺页；
5. 写保护缺页处理完成后，后续写访问可以继续执行。

实现时要注意装入过程本身也需要写目标页：空页需要清零，换入页需要从 swap 文件读入。可以先在装入阶段临时允许写入，把内容准备好；内容准备完成后，再切换到进阶策略要求的访问权限。

为了让 handler 能区分不同类型的 fault，用户态页表中可以增加必要的状态信息，例如页面是否驻留、是否已经 dirty、当前期望的访问权限等。测试时应构造读多写少的访问序列，并通过日志或 swap 写回次数说明 clean page 没有被无意义写回。